

Université du Québec en Outaouais

Département d'informatique et d'ingénierie

Sigle : INF4083

Cours : Langages de Programmation

Devoir 1 : Interpréteur lambda-calcul

Session : Automne *2026*

1.1 Professeur : Alan Davoust

DAVIDSON ANIS - ANID69360004

SOPIE DORYNE AGOUA - AGOS76360005

MAMONDE BRIAN KELOME KELM04299804

1- Introduction

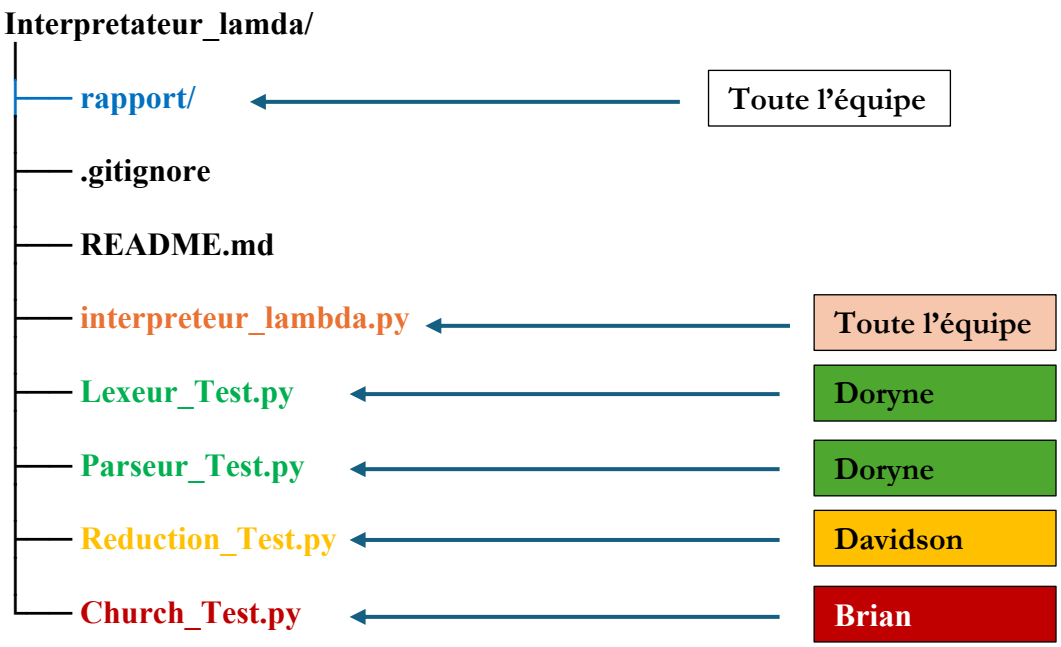
Le lambda-calcul repose sur trois éléments : les variables, les fonctions (abstractions) et les applications de fonctions. Son écriture est courte, mais l'évaluation d'une expression demande de suivre des règles de syntaxe précises et de bien gérer la substitution des variables.

Ce devoir consiste à écrire en Python un interpréteur capable de découper un texte en unités lexicales, de construire un arbre syntaxique, d'effectuer des bêta-réductions et de calculer des additions avec les entiers de Church. L'énoncé impose des noms de fonctions précis, ainsi que la remise d'un seul fichier Python accompagné d'un rapport en PDF.

Nous avons réparti le travail entre **trois membres** de l'équipe. Chacun a programmé ses fonctionnalités séparément, puis nous les avons vérifiées avec des scripts de tests et des essais en mode interactif. Les résultats présentés dans ce rapport sont ceux que notre équipe a obtenus.

2- Organisation du travail et utilisation de GitHub

2.1- Répartition



2.2- Gestion du code source

Le dépôt commun nous permet de garder tout le programme au même endroit et de suivre le travail de chacun. L'équipe a une branche principale, **main**, et des branches de travail où chaque fonctionnalité est développée avant d'être intégrée. On peut comparer les modifications grâce à l'historique des **commits** et, quand elles ont été utilisées, aux **Pull Requests**.

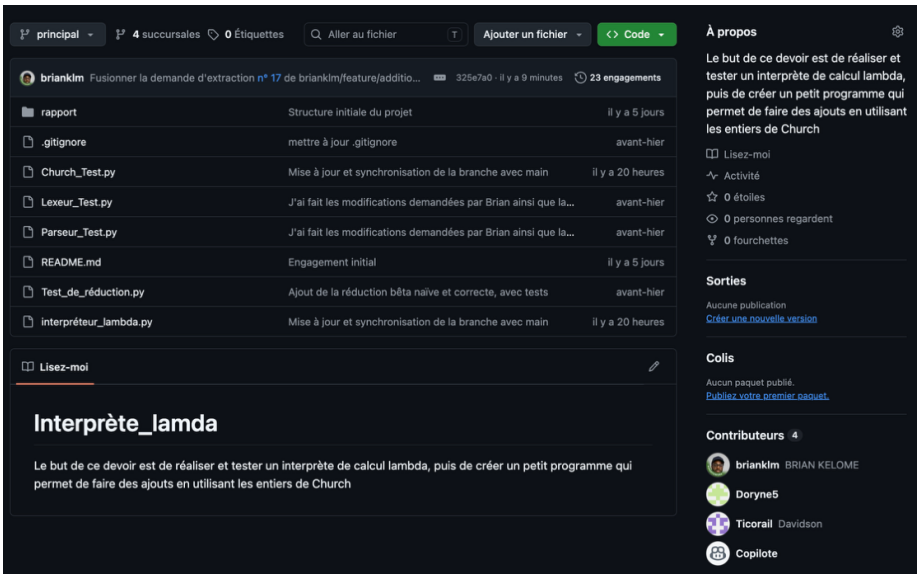


Figure 1 : Interface de dépôt GitHub

Membre	Branche GitHub	Travail réalisé	Fichiers de tests
Doryne	Lexeur-Parseur	Analyse lexicale, analyse syntaxique et affichage des arbres	Lexeur_Test.py Parseur_Test.py
Davidson	beta_reduction	Bêta-réduction naïve et correcte, normalisation et équivalence alpha	Reduction_Test.py
Brian	Addition-church	Entiers de Church, addition interface utilisateur et intégration	Church_Test.py
Équipe	main	Regroupement du code, vérification et version principale	Tous les tests

Tableau 1 : Tableau des branches et des responsabilités

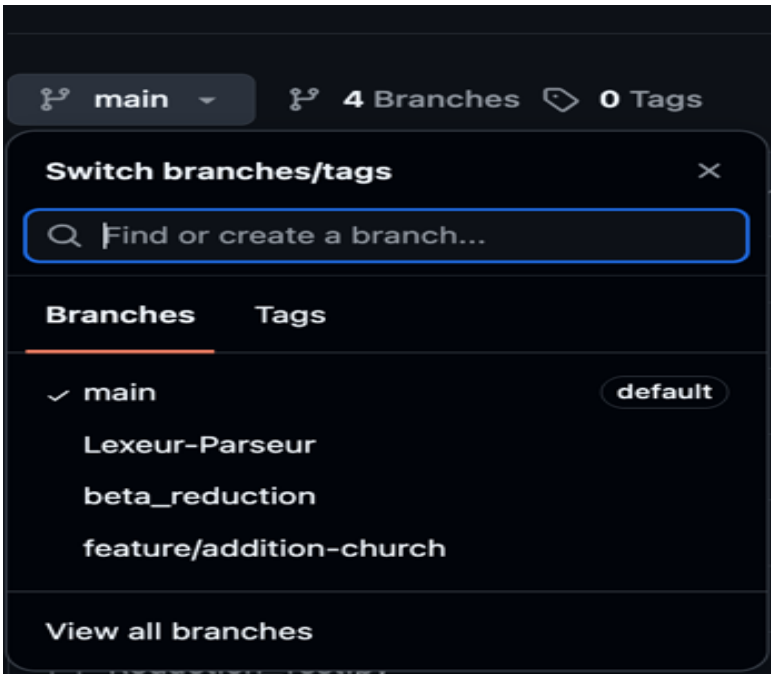


Figure 2 : Listes des Branches

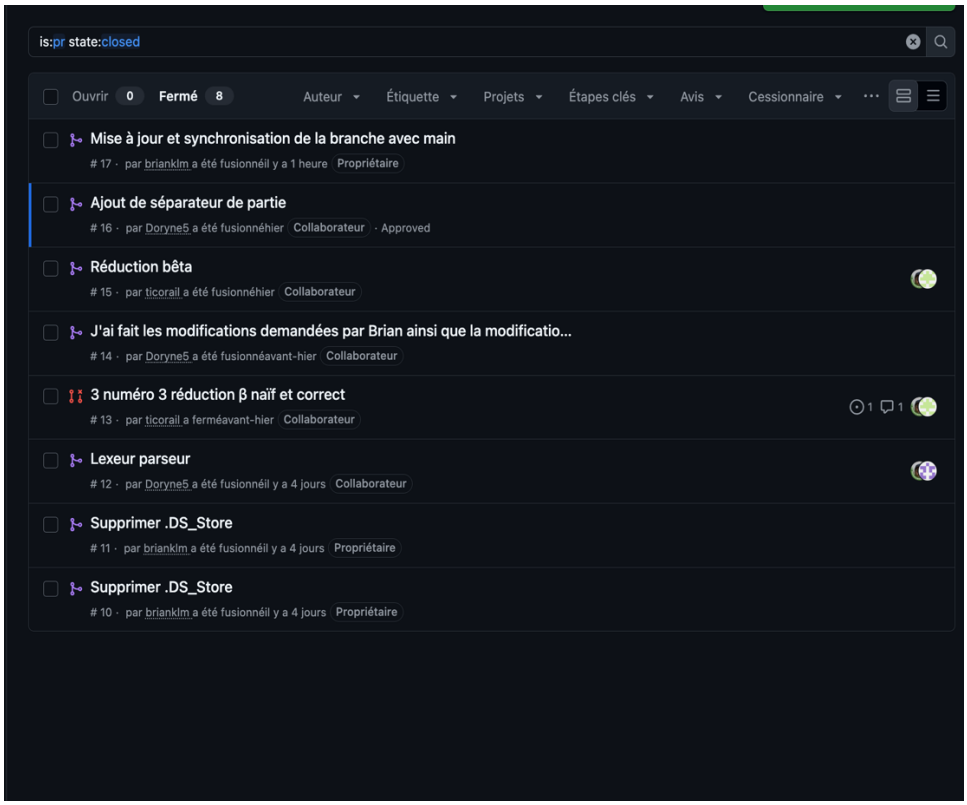


Figure 3 : Listes des Pull Requests

2.3- Livrable

Notre projet donne lieu à deux livrables, que nous déposons sur Moodle :

interpreteur_lambda.py : le fichier Python qui réunit toutes les fonctionnalités de notre interpréteur de lambda-calcul.

Le rapport : il explique comment le programme est construit, ce que nous avons réalisé, comment nous nous sommes réparti le travail et ce que les tests nous ont appris.

Au fil du développement, nous avons écrit quatre fichiers de tests pour vérifier notre travail étape par étape : **Lexeur_Test.py**, **Parseur_Test.py**, **Reduction_Test.py** et **Church_Test.py.py**

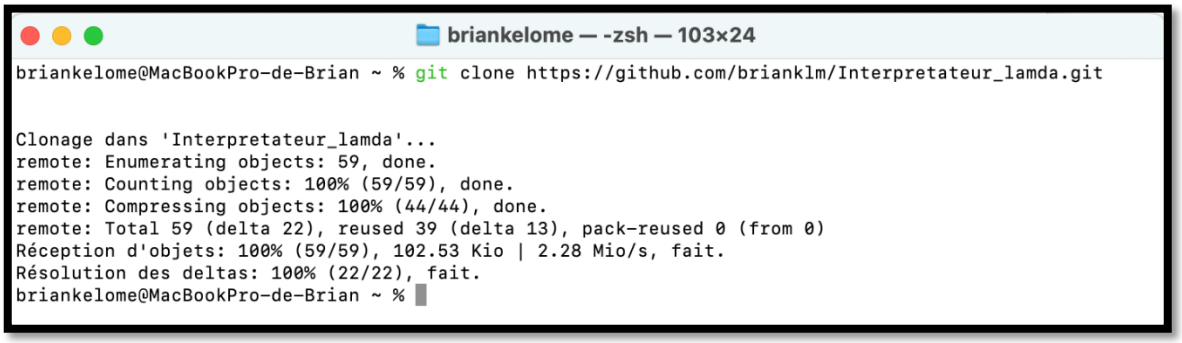
Le restent disponibles sur notre dépôt GitHub aux

https://github.com/brianklm/Interpretateur_lamda.git

pour télécharger le code source, où chacun peut voir ce que nous avons contrôlé, mais nous ne les déposons pas séparément sur Moodle. Les résultats les plus importants sont présentés et commentés dans le rapport.

Pour le télécharger localement dans votre invite de commande

Git clone https://github.com/brianklm/Interpretateur_lamda.git pour télécharger le code source



3. Architecture générale de l'interpréteur

Notre interpréteur de lambda-calcul est développé en Python dans un fichier principal nommé **interpreteur_lambda.py**.

Le programme est organisé en quatre grandes parties. Chaque partie joue un rôle précis dans son fonctionnement.

- **Partie 1** : Lexeur et parseur
- **Partie 2** : Bêta-réductions
- **Partie 3** : Entiers de Church et addition
- **Partie 4** : Programme principal et intégration

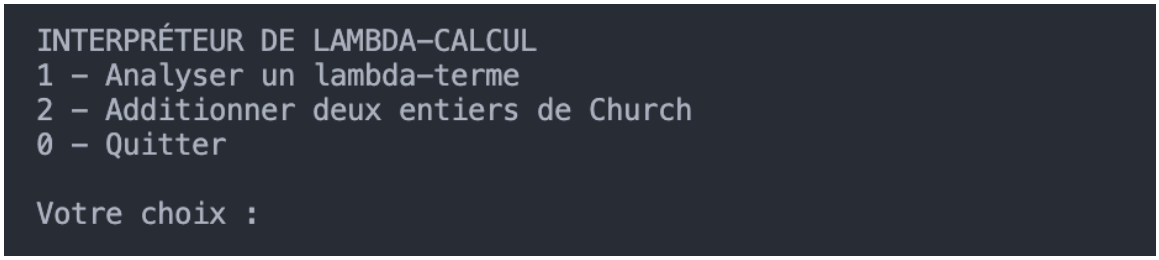


Figure 4 : Vue interface

Partie 1 : Le lexeur et le parseur ont été développés par Doryne. Ils analysent les expressions de lambda-calcul et construisent les arbres syntaxiques qui serviront aux étapes suivantes.

Partie 2 : La bêta-réduction et la normalisation sont l'œuvre de Davidson. Ses fonctions simplifient les expressions jusqu'à obtenir un résultat.

Partie 3 : Les entiers de Church ont été pris en charge par Brian. Cette partie permet de représenter des nombres et de les additionner, en s'appuyant sur les fonctions de réduction écrites par Davidson.

Partie 4 : Le menu principal a aussi été réalisé par Brian. Il permet à l'utilisateur de choisir une opération et d'en afficher le résultat.

Ces quatre parties cohabitent dans le même fichier et fonctionnent ensemble pour former notre interpréteur de lambda-calcul.

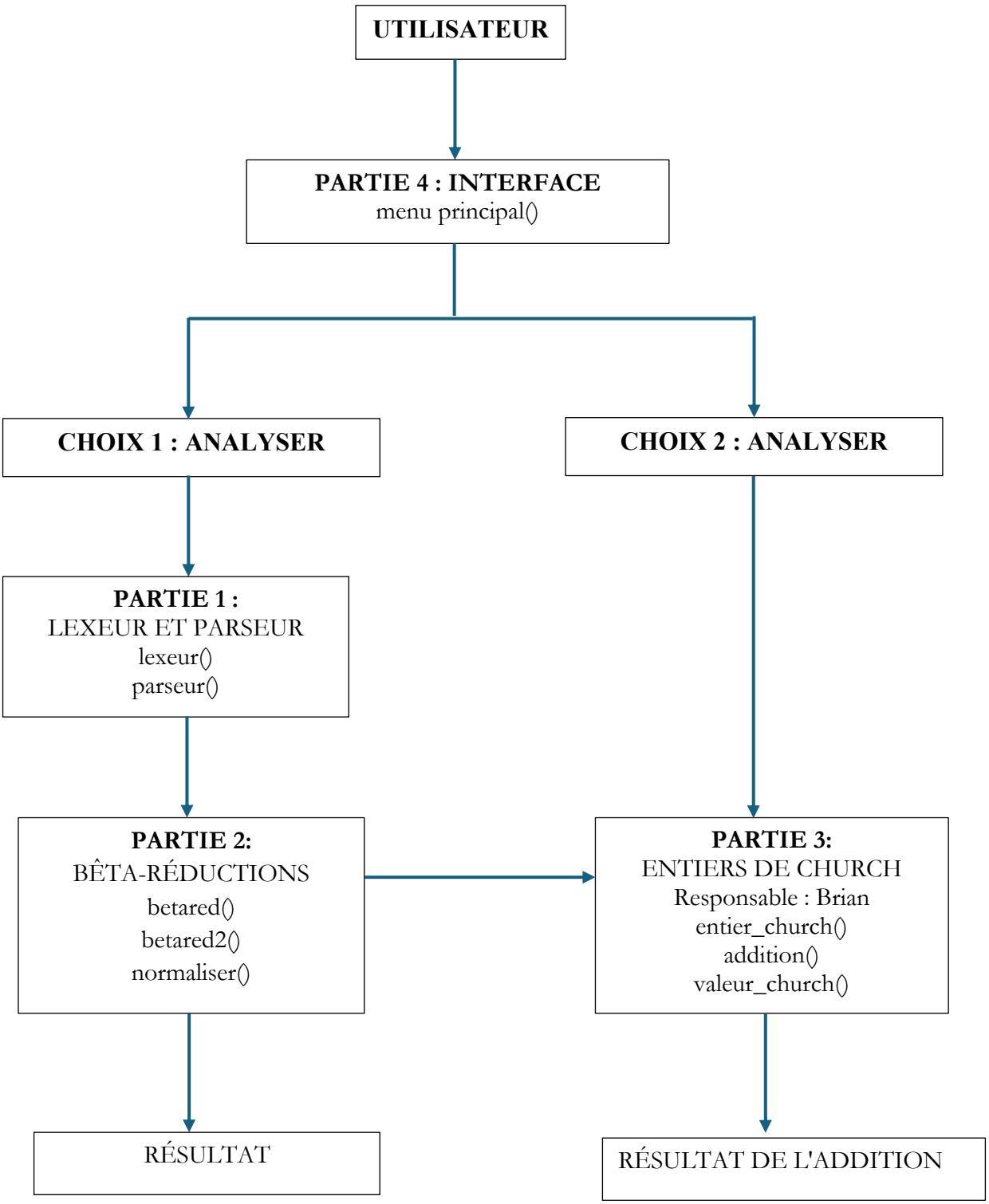


Figure 5 : Voici un schéma qui correspond aux code interpreteur_lambda.py :

4. Analyse lexicale (Lexeur et Parseur)

4.1 Analyse lexicale (Lexeur)

4.1.1 Principe

La fonction Lexeur() prend en entrée une chaine de caractère contenant les lambda-terme, qu'elle devait lire de gauche à droite et retourner une liste de Tokens qui affichait une paire (t,p). Où t est le texte du token et p le type de t. la fonction est basé sur le principe suivant :

Type de Token	Nom utiliser dans le code	Exemple de token
Symbole lambda	LAMBDA	'/'
Point	POINT	' . '
Espace	ESPACE	' '
Parenthèse ouvrante	PARO	'('
Parenthèse fermante	PARF	')'
Variable	VAR	'x', 'y', 'a1', 'adcv12', …

Tous les autres caractères qui ne sont pas contenu dans ce tableau sont considérés comme des erreurs lexicales

4.1.2 Tests et erreurs

Les différents tests sont dans le fichier python Lexeur_Test.py. Pour chacune des entrées, le programme compare les données à la liste qu'on a dans principe et sort la liste des tokens. En cas d'erreurs, le programme affiche un message d'erreurs lexicales et affiche l'élément qui pose un problème. On a les résultats des tests effectués :

```
Terme : '/x.y x'
[('/', 'LAMBDA'),
 ('x', 'VAR'),
 ('.', 'POINT'),
 ('y', 'VAR'),
 (' ', 'ESPACE'),
 ('x', 'VAR')]
```

```
 Terme : 'xx (xz  t)'
 [('xx', 'VAR'),
  (' ', 'ESPACE'),
  ('(', 'PARO'),
  ('xz', 'VAR'),
  (' ', 'ESPACE'),
  (' ', 'ESPACE'),
  ('t', 'VAR'),
  (')', 'PARF')]
```

```
Terme : ')/xt.'
[(')', 'PARF'), ('/', 'LAMBDA'), ('xt', 'VAR'), ('.', 'POINT')]
```

```
Terme : '/x.x+2'
Erreur lexicale sur le (+)
```

```
Terme : 'y2 *'
Erreur lexicale sur le (*)
```

Ces tests sont considérés comme suffisant car elle couvre tous les critères d'acceptation et de rejet émit dans l'énoncé du devoir :

- Les cas particuliers comme deux variables consécutives sans espace entre eux est considéré comme une variable
- Le Lexeur ne gère pas les erreurs de syntaxe

4.2 Analyse lexicale (Parseur)

4.2.1 Règle de précédence

Le Parseur doit respecter deux règles de précédence qui sont l'application et l'abstraction.

- L'application est déterminée dans le code par la fonction analyser_application() qui commence par lire une première variable et la place dans une variable qu'on va appeler gauche dans le code. Dans une boucle While (Tant que), si le token suivant est un espace la variable qui vient après cet espace et le place dans la variable droite. Ensuite, la variable gauche=application (gauche, droite) s'exécute. La boucle s'arrête lorsqu'aucun espace ne suit, ainsi les applications s'empilent vers la gauche.

- Dans analyser_abstraction(), le parseur reconnaît d'abord le symbole /, puis lit le nom de la variable et vérifie que le point qui la sépare du corps est bien présent. Ensuite, l'instruction corps, i = analyser_terme(a, i) est exécuté. L'analyse du corps se poursuit aussi loin que possible et ne s'arrête que devant une parenthèse fermante ou à la fin des tokens. Comme analyser_terme() permet de faire appel à la fonction analyser_abstraction(), ainsi on peut reconnaître les abstractions imbriquées comme dans (/x./y.x).

4.2.2 Tests et Erreurs

Pour les tests, les lambda termes de l'exercice1 du TD2 ont été utilisé, de plus 5 autres entrées ont été ajouté pour montrer que le code marche pour d'autre valeur. La fonction Parseur() prend en entrée un terme sous forme de texte et retourne la liste des tokens et s'il n'y a pas d'erreur de syntaxe le terme est écris comme appris en classe et la fonction affiche() affiche l'arbre syntaxique. En cas d'erreur de syntaxe, un message d'erreur syntaxique est affiché après la liste de token.

```
T2 = "/x.(/y.x) (x z)"

Séquence de Tokens :
[('/', 'LAMBDA'),
 ('x', 'VAR'),
 ('.', 'POINT'),
 ('(', 'PARO'),
 ('/', 'LAMBDA'),
 ('y', 'VAR'),
 ('.', 'POINT'),
 ('x', 'VAR'),
 (')', 'PARF'),
 (' ', 'ESPACE'),
 ('(', 'PARO'),
 ('x', 'VAR'),
 (' ', 'ESPACE'),
 ('z', 'VAR'),
 (')', 'PARF')]

Parenthèse : (λx.((λy.x) (x z)))

abstraction
├── λx
└── application
    ├── abstraction
    │   ├── λy
    │   └── variable x
    └── application
        ├── variable x
        └── variable z
```

```
T6 = "/z./x.(x./y) z "

Séquence de Tokens :
[('/', 'LAMBDA'),
 ('z', 'VAR'),
 ('.', 'POINT'),
 ('/', 'LAMBDA'),
 ('x', 'VAR'),
 ('.', 'POINT'),
 ('(', 'PARO'),
 ('x', 'VAR'),
 ('.', 'POINT'),
 ('/', 'LAMBDA'),
 ('y', 'VAR'),
 (')', 'PARF'),
 (' ', 'ESPACE'),
 ('z', 'VAR'),
 (' ', 'ESPACE')]

Erreur de syntaxe sur le .
```

```
T9 = "x y z"

Séquence de Tokens :
[('x', 'VAR'), (' ', 'ESPACE'), ('y', 'VAR'), (' ', 'ESPACE'), ('z', 'VAR')]

Parenthèse : ((x y) z)

application
├── application
│   ├── variable x
│   └── variable y
└── variable z
```

5. Entiers de Church et addition

5.1 Beta-réductions (version naïve)

Cette partie réduit le lambda-terme sans tenir compte des variables qui peuvent être capturées. La fonction normaliser répète la réduction tant que le terme contient un redex.

La fonction betared(t) fait une seule étape de réduction. Elle appelle la fonction réduire, qui cherche le redex le plus proche de la racine et le réduit avec la fonction substitution.

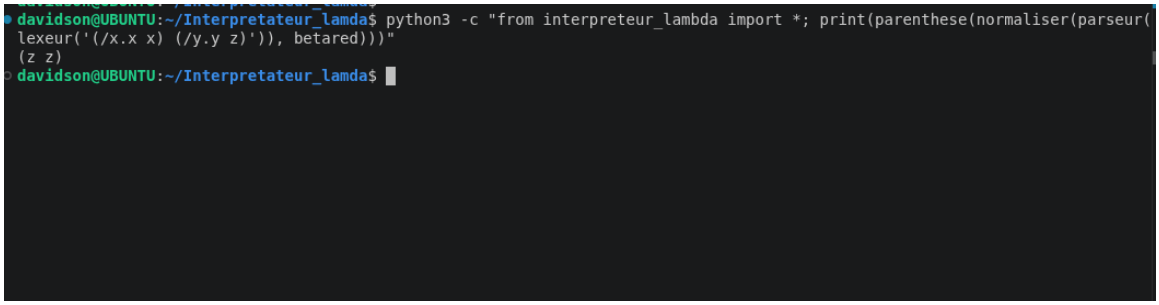
Exemple 1 : T3 = (λx.x x) (λy.y z)

Ici, **réduire** regarde s'il y a un redex à la racine, et la réponse est oui. On appelle **substitution**, qui remplace les x de x x par (λy.y z), ce qui donne (λy.y z) (λy.y z).

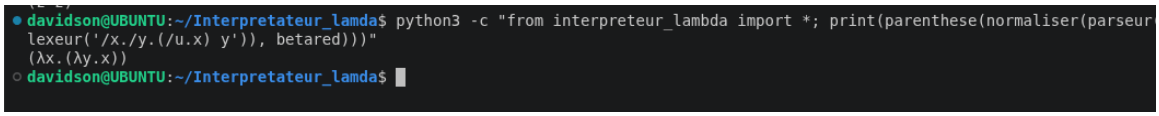
On recommence : **réduire** trouve encore un redex à la racine. On fait à nouveau la substitution, ce qui donne (λy.y z) z.

La fonction trouve encore un redex et réduit à nouveau en z z.

La nouvelle vérification de redex est fausse, donc on s'arrête ici. Le résultat z z est correct.



Exemple 2 : T4 = λx.λy.(λu.x) y

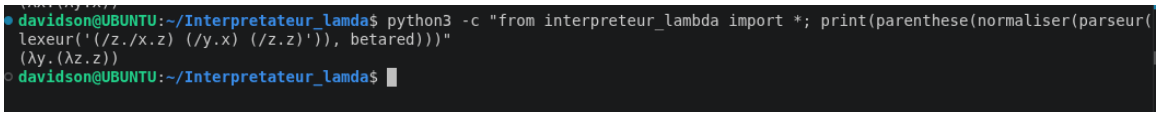


Ici, la racine n'est pas un redex, c'est une abstraction. **réduire** descend donc dans le corps, puis encore dans le corps de λy, et trouve le redex (λu.x) y.

On appelle **substitution** pour remplacer u par y dans x. Comme il n'y a pas de u dans x, rien ne change et il reste x. On obtient λx.λy.x.

Il n'y a plus de redex, donc on s'arrête. Le résultat λx.λy.x est correct.

Exemple 3 : (λz.λx.z) (λy.x) (λz.z)



Ici, la racine n'est pas un redex, car sa partie gauche est une application. **réduire** cherche donc à gauche et trouve le redex (λz.λx.z) (λy.x).

On appelle **substitution** pour remplacer z par (λy.x) dans λx.z, ce qui donne λx.λy.x. C'est là que se trouve le problème : le x de (λy.x) était libre, et maintenant il est lié au λx. La version naïve ne vérifie pas ça. On a maintenant (λx.λy.x) (λz.z).

réduire trouve un nouveau redex à la racine. On remplace x par (λz.z), ce qui donne λy.λz.z.

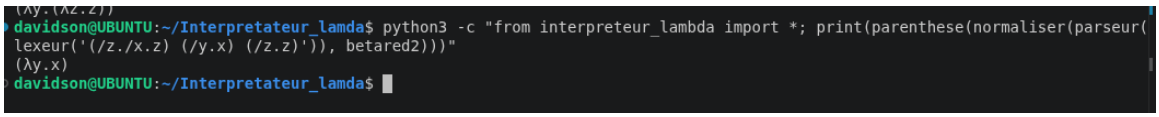
Il n'y a plus de redex, donc on s'arrête. Le résultat λy.λz.z est donc faux.

5.2 Beta-réductions correctes

Cette partie fait la même réduction que la version naïve, mais elle évite que des variables libres soient capturées. La fonction **betared2(t)** utilise la même fonction reduire pour trouver le redex le plus proche de la racine. La différence se trouve dans la fonction substitution2.
Avant d'entrer dans une abstraction λy, **substitution2** vérifie si y est libre dans le terme qu'on veut mettre à la place. Si c'est le cas, elle renomme d'abord le y lié avec un nouveau nom (y1, y2, etc) choisi par la fonction **nom_frais**, puis elle fait la substitution.

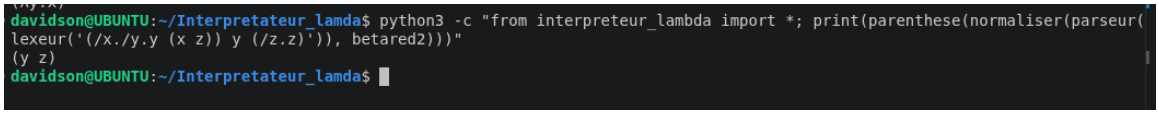
Exemple 1 : A = (λz.λx.z) (λy.x) (λz.z)
Comme dans la version naïve, reduire trouve d'abord le redex (λz.λx.z) (λy.x) et veut remplacer z par (λy.x) dans λx.z.
Mais cette fois, **substitution2** voit que x est libre dans (λy.x).
Si on substitue directement, ce x serait capturé par le λx. Elle renomme donc le λx en λx1, puis elle fait la substitution. On obtient (λx1.λy.x) (λz.z).

réduire trouve un nouveau redex à la racine. On remplace x1 par (λz.z) dans λy.x, mais comme il n'y a pas de x1 dedans, rien ne change. On obtient λy.x.
Il n'y a plus de redex, donc on s'arrête. Le résultat λy.x est correct, et le x est resté libre jusqu'à la fin, comme au départ.



Exemple 2 : B = (λx.λy.y (x z)) y (λz.z)

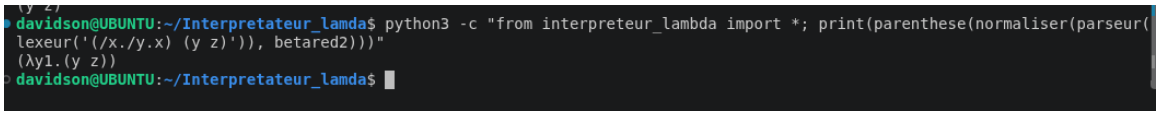
réduire trouve d'abord le redex (λx.λy.y (x z)) y et veut remplacer x par y.
substitution2 voit que la variable liée y est aussi libre dans l'argument y. Elle renomme donc le λy en λy1, ce qui donne λy1.y1 (x z), puis elle remplace x par y. On obtient (λy1.y1 (y z)) (λz.z).
réduire trouve un redex à la racine. On remplace y1 par (λz.z), ce qui donne (λz.z) (y z).
On réduit encore une fois : z est remplacé par (y z), ce qui donne y z.
Il n'y a plus de redex, donc on s'arrête. Le résultat y z est correct



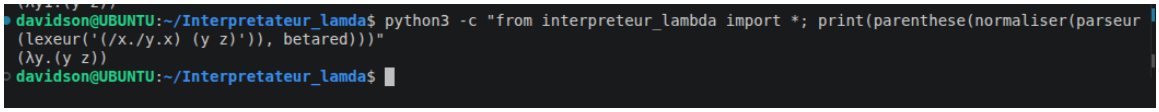
Exemple 3: T10 = (λx.λy.x) (y z)

réduire trouve le redex à la racine et veut remplacer x par (y z) dans λy.x.

substitution2 voit que y est libre dans (y z). Elle renomme donc le λy en λy1, puis elle remplace x par (y z). On obtient λy1.y z.



Avec la version naïve, on aurait obtenu λy.y z, où le y libre est capturé, ce qui est faux. Nous montrons le résultat dans l'image qui suit.



6. Entiers de Church et addition

6.1 Objectif de cette partie

L'objectif de cette partie est de représenter les entiers naturels à l'aide du lambda-calcul et d'effectuer des additions en utilisant les bêta-réductions.

Pour réaliser cette fonctionnalité, nous avons développé trois fonctions principales :

- entier_church(n) : convertir un entier naturel en entier de Church.
- valeur_church(t) : convertir un entier de Church en entier Python.
- addition(m, n) : additionner deux entiers naturels à l'aide du lambda-calcul.

Ces fonctions utilisent les structures d'arbres syntaxiques définies dans la première partie du projet ainsi que la fonction de normalisation développée dans la deuxième partie.

6.2. Représentation des entiers de Church

Les entiers de Church permettent de représenter les nombres naturels à l'aide de fonctions.

Chaque entier correspond au nombre de fois où une fonction f est appliquée à une variable x.

Entier	Représentation de Church
0	λf.λx.x
1	λf.λx.f x
2	λf.λx.f (f x)
3	λf.λx.f (f (f x))

Par exemple, pour représenter le nombre 2, nous appliquons deux fois la fonction f à la variable , cette représentation permet de manipuler les nombres sans utiliser directement les opérations arithmétiques habituelles.

6.2 Conversion en entiers de Church

La fonction **entier_church(n)** transforme un entier naturel en entier de Church.

```
457     corps = variable('x')
458
459     for _ in range(n):
460         corps = application(variable('f'), corps)
461
462     return abstraction('f', abstraction('x', corps))
463
```

La boucle applique la fonction f exactement n fois à la variable x.

Par exemple, pour n = 2,

```
591  # TEST ENTIER DE CHURCH 2
592  print(parenthese(entier_church(2)))
```

nous obtenons : **λf.λx.f (f x)**

```
(λf.(λx.(f (f x))))

INTERPRÉTEUR DE LAMBDA-CALCUL
1 - Analyser un lambda-terme
2 - Additionner deux entiers de Church
0 - Quitter

Votre choix :
```

La fonction valeur_church(t) effectue l'opération inverse. Elle compte les applications de f pour retrouver la valeur numérique.

6.3 Addition de deux entiers

La fonction **addition(m, n)** permet d'additionner deux entiers de Church sans utiliser directement l'opérateur + de Python.

```
493  def addition(m, n):
494      """Additionne deux entiers naturels par bêta-réduction."""
495
496      M = entier_church(m)
497      N = entier_church(n)
498
499      S = abstraction(
500          'n',
501          abstraction(
502              'f',
503              abstraction(
504                  'x',
505                  application(
506                      variable('f'),
507                      application(
508                          application(variable('n'), variable('f')),
509                          variable('x')
510                      )
511                  )
512              )
513          )
514      )
515
516      expression = application(application(M, S), N)
517
518      resultat = normaliser(expression)
519
520      return valeur_church(resultat)
```

Dans cet extrait, S est le successeur de Church construit précédemment dans la fonction.

Le programme convertit les deux nombres en entiers de Church, construit l'expression d'addition, puis utilise **normaliser()** pour effectuer les bêta-réductions.

```
591  # TEST ADDITION DE CHURCH
592  print("Résultat de 2 + 3 :", addition(2, 3))
```

Exemple : addition(2, 3) retourne **5**.

```
Résultat de 2 + 3 : 5

INTERPRÉTEUR DE LAMBDA-CALCUL
1 - Analyser un lambda-terme
2 - Additionner deux entiers de Church
0 - Quitter

Votre choix :
```

6.4 Tests et résultats

Les fonctionnalités ont été vérifiées à l'aide du fichier Church_Test.py.

```
43        for m, n, attendu in (
44            (0, 0, 0),
45            (0, 3, 3),
46            (3, 0, 3),
47            (1, 2, 3),
48            (2, 3, 5),
49            (3, 2, 5)
50        ):
51            resultats.append(
52                verifier(
53                    f"addition({m}, {n})",
54                    lambda m=m, n=n: addition(m, n),
55                    attendu
56                )
57            )
58
```

Lors de notre précédente vérification, **14 tests sur 14 ont réussi**.

Ces tests permettent de vérifier les conversions, les additions et certaines entrées invalides.

```
briankelome@MacBookPro-de-Brian Interpretateur_lamda % python3 Church_Test.py
Résultat de 2 + 3 : 5
RÉUSSI — encodage/décodage 0 : obtenu=0, attendu=0
RÉUSSI — encodage/décodage 1 : obtenu=1, attendu=1
RÉUSSI — encodage/décodage 2 : obtenu=2, attendu=2
RÉUSSI — encodage/décodage 3 : obtenu=3, attendu=3
RÉUSSI — encodage/décodage 5 : obtenu=5, attendu=5
RÉUSSI — addition(0, 0) : obtenu=0, attendu=0
RÉUSSI — addition(0, 3) : obtenu=3, attendu=3
RÉUSSI — addition(3, 0) : obtenu=3, attendu=3
RÉUSSI — addition(1, 2) : obtenu=3, attendu=3
RÉUSSI — addition(2, 3) : obtenu=5, attendu=5
RÉUSSI — addition(3, 2) : obtenu=5, attendu=5
RÉUSSI — entier négatif : erreur attendue détectée
RÉUSSI — entier non entier : erreur attendue détectée
RÉUSSI — terme non Church : erreur attendue détectée
============================================================
14/14 tests réussis
briankelome@MacBookPro-de-Brian Interpretateur_lamda %
```

7. Programme principal et intégration

Nous avons également réalisé l'intégration des différentes parties du programme.

La fonction traiter(s) utilise le lexeur, le parseur et les fonctions de bêta-réduction pour analyser un lambda-terme.

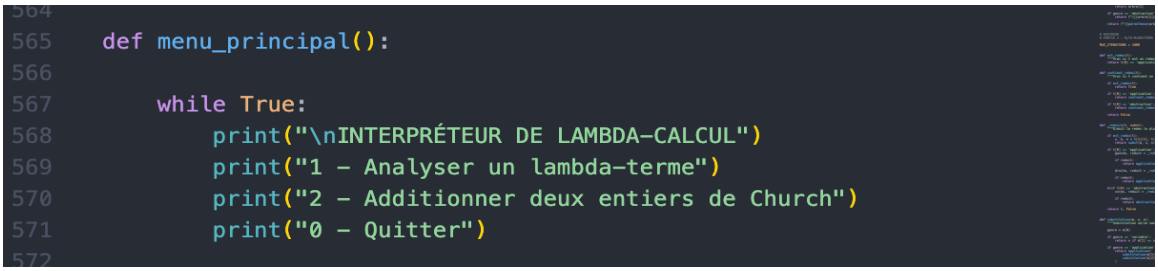
La fonction traiter_addition() demande deux entiers naturels à l'utilisateur et affiche leur somme.

```
525  def traiter(s):
526      try:
527          tokens = lexeur(s)
528
529          print("\nSéquence de Tokens :")
530          pprint(tokens)
531
532          a = parseur(tokens)
533
534          print("\nTerme :", parenthese(a))
535
536          print("\nArbre syntaxique :")
537          affichage(a)
538
539          print("\nBêta-réduction naïve :")
540          print(parenthese(betared(a)))
541
542          print("\nBêta-réduction correcte :")
543          print(parenthese(betared2(a)))
544
545          print("\nRésultat final :")
546          print(parenthese(normaliser(a)))
547
548      except (ErreurLexicale, ErreurSyntaxe) as e:
549          print("Erreur :", e)
550
```

Enfin, menu_principal () propose trois possibilités :

- **Choix 1 :** analyser un lambda-terme.
- **Choix 2 :** additionner deux entiers de Church.
- **Choix 0 :** quitter le programme.

Cette interface permet de réunir les fonctionnalités développées par les trois membres de l'équipe dans un seul programme Python.



```
564
565   def menu_principal():
566
567       while True:
568           print("\nINTERPRÉTEUR DE LAMBDA-CALCUL")
569           print("1 - Analyser un lambda-terme")
570           print("2 - Additionner deux entiers de Church")
571           print("0 - Quitter")
572
```

9 Conclusion

Le projet a permis de construire et de tester les principales composantes d'un interpréteur de lambda-calcul : tokenisation, analyse syntaxique, bêta-réductions, prévention de la capture de variables, normalisation, équivalence alpha et addition au moyen des entiers de Church.

La séparation des fonctionnalités a facilité les vérifications, tandis que l'intégration a permis de tester une chaîne d'exécution complète.

Les résultats communiqués sont encourageants : 25 tests de réduction et 14 tests de Church ont réussi automatiquement ; 14 autres cas du lexeur et du parseur ont été examinés.

Le rapport met toutefois en évidence la nécessité de vérifier le fichier source final et de documenter les tests du TD2. Les améliorations possibles comprennent davantage de tests limites et une évaluation des performances sur des termes complexes.

10. Annexe Déclaration de l'utilisation de l'IA

L'énoncé demande de nommer les outils d'IA employés, de lister tous les prompts soumis et de résumer brièvement chaque réponse. La liste complète ne peut pas être reconstruite fidèlement à partir des seuls documents disponibles. Chaque membre doit fournir son historique réel et expliquer les modifications qu'il a lui-même validées.

- **Brian** **CHAPGPT .6**

Élément	Description
Membre	Brian Kelome
Outil d'IA utilisé	ChatGPT (OpenAI)
Contexte	Le projet consiste à développer un interpréteur de lambda-calcul en Python. Ma responsabilité porte sur l'implémentation des entiers de Church, de l'addition et sur l'intégration des différentes parties du programme.
Prompt utilisé	Dans le cadre de notre projet d'interpréteur de lambda-calcul en Python, aide-moi à implémenter les entiers de Church et l'addition dans le fichier interpreteur_lambda.py. Utilise la structure d'arbre syntaxique déjà définie par mes collègues, sans modifier leurs fonctions. Implémente une fonction entière Church(n) permettant de représenter un entier naturel sous forme de lambda-terme, ainsi qu'une fonction addition(m, n) permettant d'additionner deux entiers de Church à l'aide de la bêta-réduction. Explique le fonctionnement du code étape par étape et propose des tests pour vérifier les résultats, notamment 2 + 3 = 5, 0 + 0 = 0 et 1 + 4 = 5. Aide-moi également à intégrer mon travail avec celui de mes collègues en utilisant Git et GitHub, tout en conservant leur code intact.
Résumé de la réponse obtenue	ChatGPT a fourni des explications et des propositions de code pour représenter les entiers de Church sous forme d'arbres syntaxiques, effectuer leur addition par bêta-réduction et vérifier les résultats. L'outil a également apporté une assistance pour l'intégration du code, la gestion des branches Git et la résolution des problèmes de fusion.
Fonctions développées	entier_church(n), addition(m, n) et utilisation de valeur_church(t) pour vérifier les résultats.
Résultats des tests	Vérification des opérations 2 + 3 = 5, 0 + 0 = 0 et 1 + 4 = 5. La fonction valeur_church(addition(2,3)) retourne correctement **5**.

- **Davidson** **Claude Opus 5.5**

Élément	Description
Membre	Davidson
Outil d'IA utilisé	Claude Opus 5.5
Contexte	Le fichier interpreteur_lambda.py contient déjà le lexeur, le parseur et les fonctions d'affichage.
Prompt exact utilisé	Écris dans interpreteur_lambda.py les fonctions des étapes 3 et 4 de l'énoncé : betared(t) pour effectuer une étape de bêta-réduction naïve sur le redex le plus extérieur, sans renommage ; betared2(t) pour effectuer une bêta-réduction correcte en évitant la capture des variables grâce aux variables libres et à l'alpha-renommage ; normaliser(t) pour réduire un terme jusqu'à sa forme normale avec une limite de 1000 itérations pour éviter les boucles infinies comme Omega ; et equivalent(t1, t2) pour vérifier l'équivalence de deux termes à un renommage des variables liées près. Conserve la structure d'arbre existante et le style du code. Distingue clairement la version naïve de la version correcte. Crée également un fichier Reduction_Test.py contenant des tests automatisés pour les cas simples, les captures de variables, Omega et l'équivalence.
Résumé de la réponse obtenue	L'IA a proposé l'implémentation des fonctions est_redex, contient_redex, substitution, betared, variables_libres, variables_liees, nom_frais, substitution2, betared2, normaliser et equivalent. Elle a également généré le fichier Reduction_Test.py permettant de vérifier automatiquement les différentes fonctionnalités.
Résultats des tests	**25 tests exécutés, 25 tests réussis (25/25).**
Utilisation dans le projet	Le code proposé a été utilisé pour développer et vérifier les fonctionnalités de bêta-réduction, de normalisation et d'équivalence des lambda-termes.

- Doryne **Claude Opus 5.5**

ÉLÉMENTS	DESCRIPTION
Membre	Doryne
Outil d'IA utilisé	Claude Opus 5.5
Liste des Prompt	Prompt 1 : Création du lexeur : Écris un lexeur pour les lambda-termes qui prend en entré des variables sous forme de texte et retourne la liste de tokens. Prompt 2 : Création du parseur (Écrire la fonction parseur() et affichage() . Prompt 3 : Gestion des erreurs : Indiquer où se trouve l'erreur lexicale et syntaxique. Prompt 4 : Tests automatisés (Créer un fichier de test pour le parseur termes T1 à T5 du TD2)
Réponse obtenu	Prompt 1 : Fonction lexeur(s) avec lecture des variables et détection des caractères interdits. Prompt 2 : Parseur par descente récursive, fonctions parseur, affichage, parenthese et classe ErreurSyntaxe. Prompt 3 : Classes ErreurLexicale et ErreurSyntaxe, avec des messages explicatifs. Prompt 4 : Tests comparant la forme parenthésée obtenue à la forme attendue, avec des termes valides et invalides.
Utilisation dans le projet	Prompt 1 : Base du lexeur de interpreteur_lambda.py. Prompt 2: Parseur et affichage de interpreteur_lambda.py. Prompt 3: Fonctions _erreur, _attendre et gestion des erreurs dans traiter. Prompt 4 : Base de Parseur_Test.py.